MODEL DEVELOPMENT DOCUMENT

Daily Minimum Temperature Forecasting

Melbourne, Australia | Historical data: 1 January 1981 – 31 December 1990

Document attribute	Description
Purpose	Document the end-to-end modelling process, evaluation, saved model, and Streamlit deployment design.
Primary model	Fourier seasonal features with ARIMA errors (implemented using statsmodels SARIMAX).
Comparator models	Seasonal-naive baseline, LSTM, and GRU.
Forecast interface	Streamlit app with a user-selected horizon of 1–36 months.
Status	Prepared from the project details and reported results available in the development conversation.

Prepared as a technical project report

1. Executive Summary

This project develops a daily time-series forecasting workflow for Melbourne minimum temperatures using observations spanning 1981–1990. The deliverable is a forecasting application that generates a future daily temperature path for a user-selected horizon and exports the forecast to CSV.

The dataset contains 3,650 daily rows. Three temperature entries are malformed strings and are converted to missing numeric values before time-based interpolation. Dates are parsed, observations are sorted chronologically, and the date becomes the time index. The final 365 observations (calendar year 1990) form the holdout test set; 1981–1989 is used for training.

Key model comparison

Model / protocol	MAE	RMSE	R^2
Seasonal naive / full-year-ahead	2.867	3.651	Not recorded
Fourier + ARIMA / full-year-ahead	1.950	2.540	Not recorded
LSTM / rolling one-day-ahead	1.7456	2.2710	0.6532
GRU / rolling one-day-ahead	1.7195	2.2332	0.6646
Fourier + ARIMA / rolling one-day-ahead	1.7326	2.2185	0.6690

Interpretation: Fourier + ARIMA has the lowest reported RMSE and highest reported R^2 among the rolling one-day-ahead models; GRU has the lowest MAE. The differences are small, so selection is metric-dependent. Full-year-ahead and rolling one-day-ahead results are not directly comparable because they use different forecast information.

Main deliverables

- Documented cleaning and chronological evaluation procedure.
- Saved model artifact: fourier_arima_model.pkl.
- Streamlit app that loads the saved model and produces forecasts.
- Reproducibility and deployment checklist.

2. Problem Definition and Scope

2.1 Analytical problem

Daily minimum temperature is a continuous time-series target. The project investigates whether historical temperature patterns can estimate future daily values and expose those estimates through an interactive application.

2.2 Objectives

- Clean historical observations while preserving chronological order.
- Explore trend, annual seasonality, stationarity, and missing values.
- Use a time-aware validation strategy that avoids random train/test leakage.
- Compare statistical and recurrent-neural-network approaches.
- Select a practical model and serialize it for inference.
- Provide a Streamlit app with selectable forecast horizons and CSV download.

2.3 Target and units

Target variable: daily minimum temperature. The project results use the dataset's temperature unit. The unit metadata was not independently verified for this document, so the unit should not be relabelled without checking the source dataset documentation.

2.4 Scope

- Historical data cleaning and exploratory time-series analysis.
- Baseline and model comparison on the 1990 holdout.
- Forecasting after the final observed date (31 December 1990).
- Local execution and Streamlit Community Cloud deployment.

2.5 Out of scope

- External meteorological covariates or live weather feeds.
- Formal prediction-interval calibration.
- Operational weather warnings or safety-critical use.
- Generalization claims for other cities or climate regimes.

3. Dataset Description and Data Quality

The dataset file is CP2_data_ts.csv. It contains a date field and a temperature field, with daily observations from 1981 through 1990.

Property	Project finding
Observation count	3,650 daily rows
First date	1981-01-01
Last date	1990-12-31
Expected cadence	Daily
Duplicate records	None reported in initial checks
Original null values	None reported before numeric coercion
Malformed entries	Three strings that cannot be parsed as numbers

3.1 Invalid values

Date	Raw value	Treatment
1982-07-20	?0.2	Coerce to missing; time interpolation
1982-07-21	?0.8	Coerce to missing; time interpolation
1984-07-14	?0.1	Coerce to missing; time interpolation

3.2 Annual averages

Year	Mean temperature
1981	11.52
1982	10.80
1983	11.19
1984	10.59
1985	11.14
1986	10.80
1987	10.85
1988	11.97
1989	11.26
1990	11.67

Annual averages vary across years, supporting the investigation of recurring seasonal structure and year-to-year variation. Ten annual averages alone are insufficient to establish a long-term climate trend.

4. Data Preparation and Cleaning

4.1 Transformation sequence

1. Rename source columns to date and temperature_original.
2. Parse dates using the expected day-month-year format.
3. Convert temperature strings to numeric values, coercing invalid strings to missing values.
4. Sort rows by date and reset the row index.
5. Set the date as the index.
6. Interpolate missing temperatures using time-based interpolation.

4.2 Reference implementation

```
df.columns = ["date", "temperature_original"]
df["date"] = pd.to_datetime(df["date"], format="%d-%m-%Y")
df["temperature"] = pd.to_numeric(
    df["temperature_original"], errors="coerce"
)
df = df.sort_values("date").reset_index(drop=True)
df = df.set_index("date")
df["temperature"] = df["temperature"].interpolate(method="time")
```

4.3 Why time interpolation?

Time interpolation uses the time index when estimating values between valid observations. It is a simple treatment for three isolated malformed values in an otherwise daily series. Long gaps or missingness associated with extreme events would require a more careful strategy.

4.4 Recommended quality checks

- Confirm date parsing created no NaT values.
- Check that dates are unique and sorted.
- Check daily cadence and identify missing calendar dates.
- Confirm target values are numeric and non-missing after treatment.
- Retain an audit trail of original malformed values and imputed replacements.

For strict production backtesting, document how imputation is handled at the train/test boundary so future information cannot influence a past forecast.

5. Exploratory Time-Series Analysis

5.1 Visual inspection

A time-series line plot reveals recurring within-year patterns, short-term spikes, and broad changes in level. Daily weather data are noisy, so a seasonal pattern can coexist with substantial day-to-day variation.

5.2 Annual seasonality

The series exhibits strong annual seasonality. An additive seasonal decomposition with period 365 was used as an approximate way to separate observed values into trend, seasonal, and residual components.

Observed value = Trend + Seasonal component + Residual

A 365-day period is an approximation: calendar years include leap days and the annual cycle is not exactly 365 days.

5.3 Stationarity test

The Augmented Dickey–Fuller test returned a test statistic of approximately -4.4405 and a p-value of 0.000251. At conventional significance levels, this is evidence against a unit-root null under the selected test specification. It does not prove that seasonality, structural changes, or all non-stationary behaviour are absent.

5.4 Implications

- Represent annual seasonality explicitly.
- Expect daily spikes to be difficult to predict from temperature history alone.
- Review plots alongside numerical metrics.
- Check residuals by month and year for seasonal bias.

5.5 Further EDA

- Plot the full series and individual years.
- Inspect month-of-year and day-of-year distributions.
- Review ACF/PACF plots for short-lag dependence.
- Test leap-day handling around late February.

6. Train/Test Design and Evaluation Protocol

6.1 Chronological split

The final 365 observations, corresponding to calendar year 1990, form the test set. Observations from 1981–1989 form the training period. This simulates forecasting a future period without using its values during model fitting.

Partition	Date range	Role
Training	1981-01-01 to 1989-12-31	Fit model
Test	1990-01-01 to 1990-12-31	Estimate out-of-sample performance

6.2 Two forecast protocols

Full-year-ahead: the model forecasts the held-out year from a fixed origin; actual test values are not fed in as each day advances.

Rolling one-day-ahead: the model predicts one day at a time and can use actual observations from earlier test days as they become available.

6.3 Why the distinction matters

Rolling scores can look better because the model receives fresh observations each day. These scores answer a different operational question from a fixed-origin annual forecast. Rank models only when forecast dates, origins, update policy, and metrics are aligned.

6.4 Evaluation discipline

- Use identical test dates and forecast protocols for each candidate.
- Fit scalers and learned preprocessing on training data only.
- Avoid repeatedly selecting settings against the final test year.
- Use rolling-origin validation within the training period where possible.
- Keep a final untouched test period for final evaluation.

7. Baseline: Seasonal-Naive Forecast

7.1 Concept

A seasonal-naive model forecasts a future value by reusing an observation from the corresponding position in a previous seasonal cycle. For daily annual seasonality, it can repeat the previous year's day-of-year pattern.

7.2 Purpose

The baseline tests whether a more complex model adds value beyond repeating a seasonal template. It is transparent, inexpensive, and easy to reproduce.

7.3 Reported performance

Protocol	MAE	RMSE
Full-year-ahead holdout	2.867	3.651

7.4 Interpretation

The Fourier + ARIMA full-year-ahead result (MAE 1.950, RMSE 2.540) is better than the reported seasonal-naive baseline. This suggests that a smooth seasonal representation plus autoregressive structure adds value on this evaluation.

7.5 Caveats

- A 365-day lag can misalign around leap years.
- Document how 29 February and day-of-year indexing are handled.
- Use the same forecast protocol for a fair baseline comparison.

8. Primary Model: Fourier Seasonality + ARIMA Errors

8.1 Model idea

The selected statistical approach represents annual seasonality with Fourier sine and cosine terms and models remaining serial dependence with an ARIMA error process. It is implemented using SARIMAX with no seasonal ARIMA component and with deterministic Fourier terms as exogenous regressors.

8.2 Fourier features

For elapsed day t and harmonic k, the model uses sine and cosine terms with period P = 365.25 days. Two harmonics (K = 2) produce four features: sin 1, cos 1, sin 2, cos 2.

$$\sin(2\pi kt / P) \text{ and } \cos(2\pi kt / P)$$

The epoch is fixed at 1 January 1981. The same epoch, period, number of harmonics, and feature order must be used for training and inference.

8.3 Why Fourier terms?

- Compact and smooth representation of recurring annual variation.
- Avoids a separate coefficient for every day of the year.
- Additional harmonics can represent more complex shapes but can overfit.
- A 365.25-day period approximates the annual cycle and does not fully encode calendar effects.

8.4 ARIMA error component

$$ARIMA(p, d, q) = ARIMA(2, 0, 1)$$

The model has two autoregressive terms, no ordinary differencing, and one moving-average term. The Fourier regressors represent deterministic seasonality, while ARIMA captures short-run residual dynamics.

8.5 Implementation notes

- An integer index was used for the endogenous series to avoid date-frequency forecast-index issues.
- Fourier features were generated from actual calendar dates.
- Future exogenous features must have the same four columns in the same order.
- The future feature matrix must contain one row per forecast date.

9. Model Fitting, Diagnostics, and Saved Artifact

Setting	Value
Model class	statsmodels SARIMAX
ARIMA order	(2, 0, 1)
Seasonal order	(0, 0, 0, 0)
Fourier period	365.25 days
Harmonics	2
Exogenous columns	sin_1, cos_1, sin_2, cos_2
Saved model observations	3,650
Convergence	Reported True for saved full-series fit
AIC	Approximately 16,667.345 for saved full-series fit

9.1 Training versus deployment fit

The saved deployment model was refitted on all 3,650 observations through 31 December 1990. This is appropriate after model evaluation. It is not the same fitted object as the training-only model used for the 1990 holdout.

9.2 Residual diagnostics

A thorough review should inspect residual time plots, residual distribution, ACF, and a Ljung–Box test. A low Ljung–Box p-value indicates remaining autocorrelation under the test assumptions. The project notes mention residuals fluctuating around zero with sizeable errors, but do not provide a final Ljung–Box result; no conclusion about that test is asserted.

9.3 Artifact files

- fourier_arima_model.pkl: serialized fitted model.
- model_metadata.json: model order, Fourier feature order, period, training dates, and package versions.
- CP2_data_ts.csv: source dataset used by the app.
- app.py: Streamlit application.

9.4 Serialization caution

Pickle files are version-sensitive and should only be loaded from trusted sources. Pinning dependencies reduces environment drift, but compatibility should still be tested in a clean environment.

10. Deep-Learning Comparators: LSTM and GRU

10.1 Sequence formulation

The LSTM and GRU use a 30-day lookback window to predict the next day's temperature. MinMaxScaler is fitted on the training period only and then applied to test data.

$$[X(t-29), X(t-28), ..., X(t)] \rightarrow X(t+1)$$

10.2 LSTM

Long Short-Term Memory networks use a memory cell and gates to control information retention and updates. The forget, input, and output gates help the model learn dependencies across sequences and mitigate some difficulties of ordinary recurrent neural networks.

10.3 GRU

Gated Recurrent Units use update and reset gates. Their simpler structure often requires fewer parameters than LSTM while retaining the ability to learn temporal patterns.

10.4 Evaluation results

Model	MAE	RMSE	R^2
LSTM	1.7456	2.2710	0.6532
GRU	1.7195	2.2332	0.6646

10.5 Interpretation

GRU modestly improves on LSTM across the reported metrics. Both neural models produce relatively smooth predictions and may miss sharp daily changes. Model complexity alone is not evidence of better performance.

10.6 Inference distinction

These scores use rolling one-day-ahead evaluation. They should not be interpreted as expected error for a single 12-month forecast made on 31 December 1990 without subsequent actual observations.

11. Model Comparison and Selection

Candidate	Protocol	MAE	RMSE	R^2
Seasonal naive	Full-year-ahead	2.867	3.651	—
Fourier + ARIMA	Full-year-ahead	1.950	2.540	—
LSTM	Rolling 1-day	1.7456	2.2710	0.6532
GRU	Rolling 1-day	1.7195	2.2332	0.6646
Fourier + ARIMA	Rolling 1-day	1.7326	2.2185	0.6690

11.1 Metric interpretation

- MAE: average absolute error; lower is better.
- RMSE: square root of mean squared error; large errors are penalized more heavily.
- R^2: compares squared prediction error with variation around the test-set mean; it can be negative.

11.2 Selection decision

Fourier + ARIMA is a defensible deployment choice because it is compact, interpretable, and has the best reported rolling RMSE and R^2. GRU has the lowest rolling MAE. The differences are small, so selection should be described as metric-dependent rather than a universal win.

11.3 Fairness caveat

Do not compare full-year-ahead errors directly with rolling one-day-ahead errors as if generated under identical conditions. A formal final comparison should rerun all candidates under the same fixed-origin annual forecast and, separately, the same rolling one-day-ahead protocol.

11.4 Recommended experiments

- Use rolling-origin validation across multiple training cutoffs.
- Evaluate fixed horizons of 1, 3, 6, and 12 months.
- Report errors by month and season.
- Compare with climatology and a simple autoregressive baseline.
- Add prediction intervals only after evaluating their coverage.

12. Forecast Generation and Application Logic

12.1 User workflow

1. Open the Streamlit application.
2. Select a horizon between 1 and 36 months.
3. Read the historical CSV and identify the final date.
4. Construct future daily dates.
5. Generate future Fourier features in the trained feature order.
6. Call the saved model to forecast the requested number of steps.
7. Display a chart and monthly summary and offer CSV download.

12.2 Date logic

With a final observed date of 31 December 1990, a 12-month selection is intended to forecast 1 January 1991 through 31 December 1991. The application should be checked for off-by-one errors because month offsets and inclusive end dates can affect forecast length.

12.3 Forecast limitations

• A 12-month forecast is a long fixed-origin forecast; no actual 1991 observations refresh the model.

• Forecasts reflect historical seasonal structure, not live atmospheric conditions.

• Point forecasts do not communicate uncertainty unless intervals are added and validated.

• Leap-day dates should be handled consistently in feature generation.

12.4 Output schema

Column	Meaning
date	Forecast calendar date
forecast_temperature	Model point forecast for that date

If the implementation uses different output column names, document the schema to match the actual CSV export.

13. Deployment, Dependencies, and Reproducibility

13.1 Project layout

```
temperature-forecasting/
■■■ app.py
■■■ CP2_data_ts.csv
■■■ fourier_arima_model.pkl
■■■ model_metadata.json
■■■ requirements.txt
```

13.2 Dependencies

```
streamlit
pandas
numpy
matplotlib
statsmodels
```

For stable deployment, pin statsmodels to the version used to create the pickle. Capture the local version using: python -c "import statsmodels; print(statsmodels.__version__)". Compatible versions of other dependencies should also be recorded.

13.3 Local run commands

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

13.4 Streamlit Community Cloud deployment

1. Put all required files in a GitHub repository.
2. Confirm the app, CSV, saved model, metadata, and requirements file are included.
3. Sign in to Streamlit Community Cloud using GitHub.
4. Create an app, choose the repository and branch, and set app.py as the entrypoint.
5. Deploy and inspect logs if package installation or model loading fails.
6. Test multiple horizons and the CSV download.

13.5 Security and maintenance

• Never load pickle artifacts from untrusted sources.

• Do not commit credentials or secrets.

• Verify permission before publishing the dataset.

• Version the training code and model metadata.

• Rebuild and test artifacts after major dependency changes.

14. Conclusions, Risks, and Future Work

14.1 Conclusions

The project demonstrates a complete introductory time-series workflow: data cleaning, exploratory analysis, chronological holdout, baseline comparison, statistical modelling, recurrent-network comparison, serialization, and interactive deployment.

The evidence supports Fourier seasonality with ARIMA errors as a reasonable deployment choice when RMSE, R^2, interpretability, and operational simplicity are considered. GRU has the lowest reported rolling MAE. Because metrics are protocol-dependent, a stronger conclusion requires a fully aligned comparison.

14.2 Key risks

Risk	Potential impact	Mitigation
Protocol mismatch	Misleading model ranking	Rerun all candidates with identical forecast origins and dates
Long-horizon uncertainty	Point forecasts appear too certain	Add validated intervals and explain limitations
Calendar mismatch	Seasonal phase errors near leap years	Test calendar features and leap-day behaviour
Artifact incompatibility	App fails to load model	Pin versions and test in a clean environment
Historical-only inputs	No response to current weather	Label as a historical time-series demonstration

14.3 Future enhancements

- Run rolling-origin backtesting across multiple years.
- Compare month/day climatology and ETS/SARIMA alternatives.
- Tune Fourier harmonics using training-period validation only.
- Add prediction intervals and assess interval coverage.
- Improve calendar and leap-year handling.
- Add automated tests for schema, date continuity, forecast length, and finite outputs.
- Show model assumptions, training date, metric definitions, and a live-weather disclaimer.

14.4 Final statement

This document records the project as developed based on the available project notes and reported results. Before formal submission, rerun the notebook from a clean kernel, verify every metric against saved outputs, confirm package versions, and update implementation details that differ from the final app and training notebook.

Appendix: Reproducibility checklist

- Dataset row count and date range verified.
- Malformed numeric values documented and treated.
- No random shuffling in the time-series split.
- Scalers fit only on training data for neural networks.
- Fourier epoch, period, harmonics, and feature order fixed.
- Saved model loads in a clean environment.
- 12-month forecast has 365 daily rows for calendar year 1991.
- CSV download and charts work locally and in cloud deployment.
- All metrics labelled with their forecast protocol.